

Reviewer Pack — Minimal Rank of Hodge Integrals over Boolean Function Output...

Entry 8abf61884d0e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 05:04:43 UTC

Minimal Rank of Hodge Integrals over Boolean Function Output Complexity

Entry ID: 8abf61884d0e **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-26 05:04:43 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Hodge Theory) **Field B** (complexity object): Complexity Theory: Boolean Function Output Complexity

Statement:

{'para1': 'For a given Boolean function $f: \{0,1\}^n \rightarrow \{0,1\}$, the Hodge integrals of its characteristic polynomial P_f over the complex projective space are non-negative real numbers.', 'para2': 'Conjecture: The minimal rank among these Hodge integrals is upper bounded by a polynomial in n , i.e., there exists a constant c such that for all Boolean functions f , the rank of the vector space spanned by its Hodge integrals is at most cn .', 'para3': 'This bound holds with high probability for random Boolean functions.'}

Rationale (proposer's reasoning):

{'para1': 'Hodge theory provides a rich algebraic invariant that may capture subtle aspects of the structure of Boolean functions, potentially revealing non-trivial relationships between their complexity and geometric properties.', 'para2': 'By considering the output complexity of Boolean functions, this conjecture aims to bridge the gap between algebraic geometry and computational complexity, which could lead to new insights into the nature of efficient computation.', 'para3': 'If true, it would suggest that certain geometric structures can be exploited to design more efficient algorithms.'}

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 89bd3a63c5b30340

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a Boolean function f , if the minimal rank of its Hodge integrals is at most cn with probability greater than or equal to 0.95 over 30 seeds, then support the conjecture.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - intitle:Hodge Theory AND complexity theory Boolean function - minrank Hodge integral characteristic polynomial AND Boolean function output complexity - Hodge integrals upper bounded polynomial n Boolean function

Top relevant hits considered: - [http://arxiv.org/abs/hep-th/9707234v2] Variational Approach to Quantum Field Theory: Gaussian Approximation and the Perturbative Expansion around It - [http://arxiv.org/abs/chao-dyn/9311011v2] The Great Inequality In A Hamiltonian Planetary Theory - [http://arxiv.org/abs/hep-th/9201009v1] Large-Small Equivalence in String Theory - [http://arxiv.org/abs/2009.05003v2] Strong approximation of Gaussian β -ensemble characteristic polynomials: the edge regime and the stochastic Airy functi - [http://arxiv.org/abs/2412.05017v5] Reduction from the partition problem: Dynamic lot sizing problem with polynomial complexity - [http://arxiv.org/abs/1112.4411v2] On the Complexity of the Generalized MinRank Problem - [http://arxiv.org/abs/2510.17487v1] Directional Search for Persistent Gravitational Waves: Results from the First Part of LIGO-Virgo-KAGRA's Fourth Observin - [http://arxiv.org/abs/1411.4413v2] Observation of the rare $B_s^0 \rightarrow \mu^+ \mu^-$ decay from the combined analysis of CMS and LHCb data

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        # Find pivot in column i
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]

        # Eliminate entries below pivot
        for j in range(i+1, n):
            factor = -A[j][i] / A[i][i]
            for k in range(n):
```

```
A[j][k] += factor * A[i][k]
```

```
def rank(A):
    n = len(A)
    r = 0
    for i in range(n):
        if all(abs(A[i][j]) < 1e-9 for j in range(r)):
            continue
        for j in range(r, n):
            A[i], A[j] = A[j], A[i]
            break
        r += 1
    return r

def characteristic_polynomial(f):
    n = len(f)
    A = [[0]*n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            A[i][j] = f[(i+j) % n]
    det = 0
    for p in itertools.permutations(range(n)):
        sign = (-1)**sum(i < j for i, j in zip(p, sorted(p)))
        prod = 1
        for i in range(n):
            prod *= A[i][p[i]]
        det += sign * prod
    return det

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    f = [random.choice([0, 1]) for _ in range(n)]

    det = characteristic_polynomial(f)
    if det == 0:
        return {
            "metric_name": "rank",
            "metric_value": None,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }

    H = [[0]*n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            H[i][j] = Fraction(1, (i + j + 2) * (i + j + 3))

    rank_H = rank(H)

    return {
        "metric_name": "rank",
        "metric_value": rank_H,
        "instances_tested": 1,
        "conjecture_holds": rank_H <= n,
```

```

        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all(r["conjecture_holds"] for r in results):
        mean_value = sum(r["metric_value"] for r in results) / len(results)
        std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
        support_fraction = 1.0
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        counterexample = "mapping_undefined"
        mean_value = None
        std_value = None
        support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)

    print(f"RESULT: {'SUPPORTED' if all(r['conjecture_holds'] for r in results) else 'FALSIFIED'}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it was unable to complete the required calculations to either support or falsify the conjecture. | next: Re-run the test with increased time limits and ensure that it completes successfully.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11915
2	preregistration	ollama_remote	glm4:latest	0	0	5473
3	novelty	ollama_remote	glm4:latest	0	0	4544
4	novelty	ollama_remote	glm4:latest	0	0	5904
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19345
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10116
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8579
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10138
9	judge	ollama_remote	glm4:latest	0	0	12149

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 88163 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/8abf61884d0e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/8abf61884d0e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/8abf61884d0e.tar.gz` (if generated)